

Sistema Promudas

Estado do Projeto · Plano de Refatoração · Offline e Rede

Revisão de memória + código + processo de negócio · 16/06/2026

Este relatório consolida o estado atual do software de PDV/encomendas do viveiro, comparando o que já está implementado com o processo de trabalho atual (em papel), apontando melhorias e decisões em aberto. A Parte II traz um plano de refatoração faseado; a Parte III analisa a viabilidade de operação offline e as opções de rede/segurança.

1. O que o sistema é (e como mapeia o processo atual)

Hoje o fluxo é manual: ficha manuscrita (cliente + pedido + pagamento na hora) gera 3 vias (cliente, vendedor, administração). A administração recopia tudo em Excel e caderno, e anota as entregas do dia à parte. O problema central é que tudo fica preso ao cliente, sem separar os diferentes pedidos.

O software ataca exatamente isso. O modelo de dados implementado já é:

cliente !' pedido (itens) !' entregas (itens) !' pagamentos

Cada pedido é uma entidade própria, com seus itens, entregas e pagamentos. Isso resolve a dor principal: um mesmo cliente pode ter vários pedidos distintos, cada um com saldo e status próprios — o salto do "caderno por cliente" para "pedido rastreável". O PDV reúne, numa só tela, o que a ficha de papel faz: dados do cliente + pedido + pagamento na hora.

2. O que JÁ está implementado e funcionando

Backend (Node + Express + Prisma/MySQL, porta 6072) — completo e integrado

- CRUD de clientes, produtos, pedidos, entregas, pagamentos, formas de pagamento, contas e locais de entrega
- Autenticação JWT + bcrypt
- Cálculo automático de status de pagamento (Pendente/Parcial/Pago/Crédito) e de entrega (Pendente/Parcial/Entregue)
- Geração de PDF do pedido (substitui uma via da ficha) e PDFs de relatórios
- Migrations Prisma organizadas para deploy via git no servidor Ubuntu

Frontend (Flutter) — dois módulos pós-login

Área	Estado
PDV (tela de venda)	Completo — busca cliente (F5), carrinho com edição inline, desconto/acréscimo, pagamento dividido (F12), cadastro rápido de cliente (Ctrl+C)
Entregas	Registrar entrega com saldo por produto, local de saída, motorista/placa; consulta de entregas registradas
Pedidos	Lista, detalhes, editar pedido, registrar/editar/excluir pagamento, emitir PDF
Clientes	CRUD, busca server-side, pedidos pendentes por cliente, saldo de crédito
Relatórios	Pagamentos e Pedidos, ambos com filtros e PDF
Configurações	Produtos, formas de pagamento, tema escuro, tamanho de fonte global
Consulta (hub)	Agrupa clientes / pedidos / entregas / pagamentos sem conta

Features de negócio que já vão além do papel

- Pagamento parcial e entrega parcial — o objetivo declarado do projeto, de ponta a ponta
- Crediário (formas com pagamento posterior) que não conta como recebido

- Conta do pagamento — para qual conta/pessoa o dinheiro entrou; com fluxo de "conta definida depois" (dinheiro)
- Parcelamento (crédito até 6x)
- Nota fiscal manual por pagamento (controle, sem integração SEFAZ)
- Pagador diferente do cliente (nome + CPF)
- Crédito do cliente quando um pedido é editado para menos
- Datas com hora (concluído em 15/06/2026)
- Sino de notificações no drawer (hoje só "pagamentos sem conta")

3. O que pode ser melhorado / pendências

Pendências técnicas (limpeza)

- Remover arquivos vazios (sqlite_helper.dart, cliente_repository.dart) e pastas mortas em core/
- Tornar data_pagamento obrigatório no schema e remover o fallback em relatorioService.js
- Decidir sobre colunas redundantes: pedidos.status_geral, entregas.status_entrega, pagamentos.recebedor
- Padronizar campo "detalhe" nas respostas de erro; mover geração do JWT para o authService
- Teste end-to-end pendente: datas-com-hora e fluxo de "pagamentos sem conta" (ainda não validado no app)

Melhorias de produto

- Redesenhar as telas de relatório (a UI; os PDFs já estão bons)
- Gestão de usuários + permissões por módulo (hoje usuarios é isolado, só login)
- Evoluir o módulo de entrega (escopo a detalhar)
- Sino como centro de notificações geral (hoje específico de uma coisa)

Gaps em relação ao processo real (vale conversar)

- Estoque — produtos só tem nome/preço/ativo; não há controle de quantidade nem baixa na venda/entrega
- Rastreo de vendedor — o pedido não registra quem foi o vendedor (usuarios não tem vínculo com pedidos)

4. Atenção estratégica: transição papel !' digital

A decisão de não remover o papel num primeiro momento é sensata. O risco da fase de convivência é a divergência: a ficha registra uma coisa e o sistema outra. Duas recomendações concretas:

- O PDF do pedido pode substituir uma das três vias desde já — ele tem cliente, itens, totais, pagamentos e crediário, reduzindo a recópia manual sem abandonar o papel de uma vez.
- A digitação na administração (Excel + caderno) é o trabalho que o sistema elimina. O ganho maior aparece quando a administração consulta relatórios/PDFs em vez de recopiar — por isso vale priorizar deixar as telas de relatório boas para que a administração confie nelas e largue o Excel.

5. Resumo executivo do estado

O projeto está maduro e funcional. O núcleo do objetivo — digitalizar a ficha, acelerar o processo e separar pedidos por cliente — já está pronto e integrado, incluindo as partes difíceis (pagamento e entrega parciais, crediário, contas). O que resta é polimento (limpeza de código, redesenho dos relatórios, testes finais) e decisões de escopo sobre temas que o papel atual tem mas o software ainda não modela: estoque, rastreo de vendedor e permissões por perfil.

PARTE II — Plano de Refatoração

Levantamento feito comparando memória + código real (evidências por arquivo:linha). Objetivo: reduzir dívida técnica SEM alterar comportamento. Como o app ainda tem testes end-to-end pendentes, a regra de ouro é refatoração pura e incremental — uma fase por vez, validando entre cada uma (flutter analyze / node --check / smoke run) e atualizando as docs de arquitetura. Abordagem escolhida: incremental, por frente.

6.1. Fase 1 — Backend: limpeza + erros (baixo risco)

- Middleware central de erro (src/middlewares/errorHandler.js) registrado após as rotas; controllers passam a usar next(erro). Corrige a inconsistência: uns retornam sempre 500 (clienteController:17, produtoController:16), outros usam erro.status||500 (pagamentoController:9).
- Usar BusinessError de forma consistente nos services que ainda não usam (cliente, produto, auth, pedido). Padronizar a resposta de sucesso.
- Mover geração do JWT de authController:20-38 para authService.
- Extrair selects/queries repetidos do Prisma: PAGAMENTO_SELECT/PEDIDO_INCLUDE (duplicados em pedidoService:57-72 e :182-197) e formaPagamentoService.listarPosteriores() (mesma query em 8 lugares).

6.2. Fase 2 — Frontend: limpeza + erros (baixo risco)

- Apagar código morto: sqlite_helper.dart, cliente_repository.dart (vazios) e pastas core/mock, core/network, core/database. Remover sqflite do pubspec. Atualizar ARQUITETURA.md.
- Helper central de erro/feedback (core/utils/api_feedback.dart) com extrairErroApi() + mostrarErro/mostrarSucesso(), substituindo os _extrairErro/_mostrarErro duplicados (pedidos_screen:379-389, modal_entrega:200-206 e ~29 usos de showSnackBar).

6.3. Fase 3 — Consolidar UI compartilhada (risco médio)

- Mover ChipStatus (lista_pedidos:105-142) para core/widgets/chip_status.dart com um único mapa de cor; eliminar as variações privadas _ChipStatus e _PillStatus (relatorio_pedidos:441-475, 576-598).
- Centralizar formatação em core/utils/formatadores.dart: formatarMoeda() para os ~96 "R\$...toStringAsFixed(2)" inline e reaproveitar formatarDataHora (já em seletor_data_hora.dart) eliminando as cópias locais de _formatarDataHora.

6.4. Fase 4 — Cores via tema (112 ocorrências, esforço alto)

- Substituir Colors.green[x]/grey[x] hardcoded por Theme.of(context).colorScheme — conserta o dark mode e cumpre o CLAUDE.md.
- Centralizar os acentos semânticos permitidos (azul=desconto, laranja=acrécimo, vermelho=erro) num core/theme/cores_semanticas.dart, em vez de espalhar literais.
- Maiores violadores: modal_pagamento (17), relatorio_pedidos (17), formulario_venda (15), detalhes_cliente (14), detalhes_pedido (9).

6.5. Fase 5 — Quebrar arquivos grandes (maior risco, por último)

- Dividir os 5 arquivos >600 linhas, um por sessão, mantendo um-widget-por-arquivo:
- detalhes_pedido.dart (740) !' extrair lista de pagamentos + helpers de cálculo.
- formulario_venda.dart (678) !' extrair gerenciador de controllers por item + _DialogAjuste.
- modal_pagamento.dart (655) !' extrair lógica de parcelas/validação para um controller.
- venda_screen.dart (652) !' extrair atalhos de teclado e a lógica de salvar edição.
- relatorio_pedidos_screen.dart (598) !' extrair bloco de filtros e cards de resultado.

6.6. Deferidos (decidir depois)

- Factory para services de catálogo (conta/locais/forma quase idênticos no back; 5 services de catálogo no front).
- Unificar `recalcularStatusPedido` × `recalcularStatusEntrega` (regras diferentes — cuidado para não acoplar).
- Biblioteca de validação (zod) no backend — adiciona dependência; decidir em conversa antes.

Resumo das fases

Fase	Frente	Risco
1	Backend: erros + JWT + selects repetidos	Baixo
2	Frontend: código morto + helper de erro	Baixo
3	UI compartilhada: ChipStatus + formatação	Médio
4	Cores via tema (112 pontos)	Alto (esforço)
5	Quebrar 5 arquivos grandes	Alto

O progresso desta refatoração é rastreado numa memória persistente (`refatoracao-plano`), atualizada ao fim de cada fase, para conduzir o trabalho ao longo de várias sessões.

PARTE III — Operação Offline e Rede

Análise da ideia de "rodar offline no Windows, sempre sincronizando com o servidor", e das opções de rede/segurança relacionadas. Status: DECISÃO EM ABERTO — por ora, apenas entendimento; nada será implementado.

7.1. "Offline com sync" é um espectro, não uma coisa só

- Nível 1 — Resiliência mínima: não perder uma venda em digitação se a conexão piscar (rascunho local + retry). Barato (dias).
- Nível 2 — Venda offline: criar pedido/pagamento sem rede e sincronizar depois; o resto exige online. Custo médio.
- Nível 3 — Offline-first total: tudo funciona sem rede, com banco local espelhado e sync bidirecional. Caro (semanas), alto risco.

7.2. Vantagens

- Continuidade de venda se a rede/servidor cair — o operador não trava no balcão.
- Latência menor nas leituras (catálogo, clientes) se vierem de um cache local.
- Robustez a quedas momentâneas — uma venda em andamento não se perde por um pico de rede.
- Abre caminho para cenários futuros (PDV em notebook/tablet, servidor na nuvem, filiais).

7.3. Desvantagens

- Complexidade alta e permanente — passam a existir "duas verdades" (local e servidor) que precisam convergir; custo vive no código pra sempre.
- É um sistema financeiro (pagamentos, crediário, saldo_credito, saldo devedor) — sincronizar dinheiro é o caso mais perigoso de divergência.
- Toda a validação/cálculo vive só no servidor hoje; offline, ou se replica essa lógica no cliente (duplicação arriscada) ou o operador só descobre rejeições DEPOIS de sincronizar.
- Mais superfície de bug e de teste — e o app ainda tem testes e2e pendentes do que já existe.

7.4. Riscos (os que realmente importam)

- IDs do servidor (autoincrement, sem UUID): um pedido criado offline não tem o número real e ele MUDA ao sincronizar — quebra a correspondência papel! digital (a via do cliente teria nº diferente do sistema).
- Conflito de concorrência: dois operadores offline mexendo no mesmo pedido/cliente; ao sincronizar, saldo_credito e status_pagamento recalculados pelo servidor podem não bater com o que cada tela mostrou. Em dinheiro, é grave.
- Validação postergada: "saldo insuficiente" / "valor excede o saldo devedor" só são checados no servidor; offline a venda parece ok e pode falhar na sincronização.
- Token JWT expira em 8h e fica só em memória: offline prolongado ou reiniciar o app = sem token para sincronizar. Precisaria persistir credenciais/token em disco.

7.5. Dificuldade de implementação — ALTA (não é "só adicionar cache")

O código hoje não tem nenhuma base para offline:

- Zero persistência local: `sqlite_helper.dart` vazio; `sqlite` no pubspec mas nunca usado (marcado para remoção); dados são `Map<String,dynamic>` crus da API, sem classes de domínio.
- `connectivity_plus` está declarado mas nunca é usado — não há nem detecção de online/offline.
- 17 endpoints de escrita chamados inline dentro das telas, sem camada de repositório — cada um teria que gravar local + enfileirar sync.
- Lógica server-authoritative (transações Prisma criando pedido+itens atomicamente, recálculos, validações)

teria que ser espelhada ou tratada na reconciliação.

Estimativa honesta: o offline-first TOTAL é uma refatoração substancial do frontend (~50–70% do código tocado), com SQLite local, modelos de domínio, fila de sincronização, IDs por UUID e resolução de conflitos — semanas de trabalho, numa área (dinheiro) onde bug custa caro. Os níveis 1 e 2 são bem mais baratos (dias / médio) e o pré-requisito de centralizar as escritas casa com o plano de refatoração.

7.6. Rede e segurança: Cloudflare × ZeroTier × LAN física

Setup atual: acesso ao servidor via Cloudflare (provável Cloudflare Tunnel). A favor: o túnel é outbound, então não exige abrir portas no roteador, e ainda dá WAF/DDoS/ocultação do IP. O risco: o serviço fica publicamente alcançável pela internet — a única barreira passa a ser o JWT.

& Alerta de segurança (tratar logo, esteja o servidor onde estiver)

- POST /api/auth/register é PÚBLICO. Com o servidor exposto pela internet, qualquer um pode criar uma conta e obter um JWT válido com acesso total (clientes, pedidos, pagamentos). Desabilitar o register público ou restringi-lo a admin.

Critério	Cloudflare Tunnel	ZeroTier
Exposição pública	Sim — alcançável por qualquer um (mitigável com Cloudflare Access)	Não — só dispositivos autorizados na rede virtual
Modelo	Publica um serviço na web	Rede privada virtual (LAN sobre internet), criptografada ponta-a-ponta
Superfície de ataque	Maior (endpoint público)	Mínima (nada exposto)
Setup nos clientes	Nada (é só uma URL)	Cliente ZeroTier instalado e autorizado em cada máquina
Para sistema financeiro	Exige endurecer (Access, tirar /register)	Postura de segurança bem melhor

7.7. O ponto que conecta tudo

Nem Cloudflare nem ZeroTier resolvem o "offline": os dois são overlays sobre a internet — se a internet da loja cair, os dois caem junto. A ÚNICA topologia que sobrevive a uma queda de internet é uma LAN física de verdade: servidor e máquinas de PDV no mesmo roteador/switch, com IPs privados, conversando sem passar pela internet.

Ou seja: se o objetivo de fundo é "continuar vendendo quando a conexão cair", a solução mais barata e robusta NÃO é programar offline-first — é colocar o servidor na mesma rede física das máquinas. Aí o dia a dia roda na LAN local (imune a queda de internet), e o ZeroTier (ou Cloudflare) entra só para acesso remoto (ex.: consultar relatórios de casa). Segurança máxima, latência mínima, sem reescrever o app.

Recomendação

- Curto prazo: arquitetura de rede — servidor na LAN física local + ZeroTier para acesso remoto; e fechar o /register público.
- Só reconsiderar offline-first se de fato surgir cenário de máquina sem rede (PDV móvel, filial sem servidor próprio).
- Se quiser algum offline, começar pelo Nível 1 (resiliência mínima) — resolve ~90% da dor real com pouco código.